

Minecraft Agent Framework

My name is Joel Chacón López, and this is my project for the TAP course on frameworks in Minecraft. Welcome to the **Minecraft Agent Framework**, an educational project that implements interactive agents in Minecraft, each with unique functionalities. This framework is designed to explore advanced programming concepts like functional and reflective programming in a fun and practical environment.

Table of Contents

- Description
- Requirements
- Installation
- Project Structure
- How to Run the Bots
- Bot Descriptions
- Unit Tests
- Functional and Reflective Programming
- Contributing

Description

This project implements interactive Minecraft bots with unique features such as insulting players, building structures, placing TNT, and answering questions (defined answers, not implemented with generative intelligence yet). It serves as a hands-on demonstration of advanced programming paradigms.

Requirements

To run this project, ensure you have:

- **Minecraft Pi Edition** or a Minecraft server compatible with the *mcp.i* API.
- **Python 3.8+** installed.

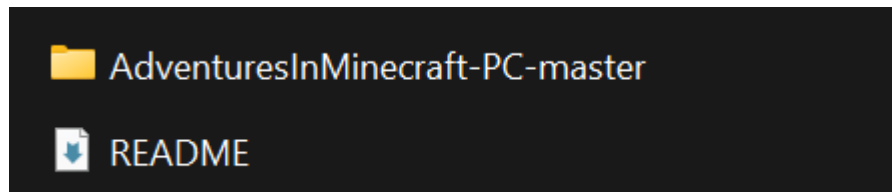
Installation

1. Clone this repository:

```
git clone https://github.com/your-username/MinecraftAgentFramework.git
```

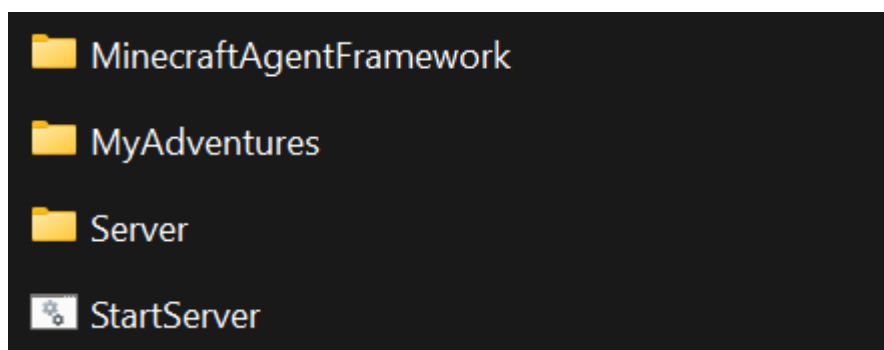
```
cd MinecraftAgentFramework
```

How to get it running:



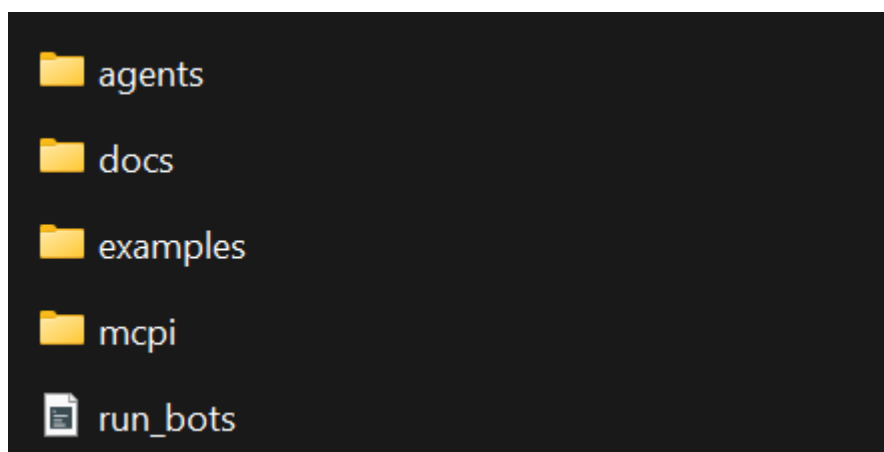
(You are here right now)

Now you need to enter the folder *AdventuresInMinecraft-PC-master*:



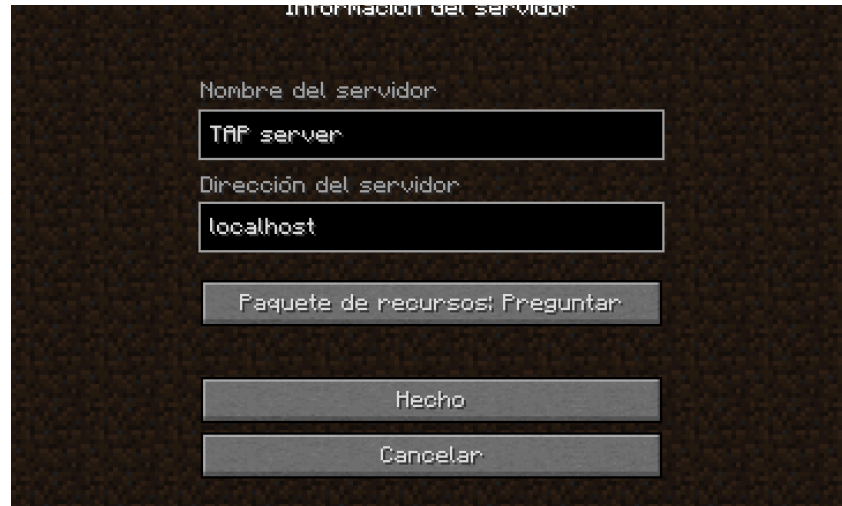
Here, use a terminal to start the server: **./StartServer**.

And now enter the *MinecraftAgentFramework* folder:



The file shown below, **run_bots.sh**, will be used to test the agents I have implemented. We will discuss them later.

To test the mentioned agents, we will open Minecraft and add a server with the address **localhost**, as shown in the following image:



After this, we are able to check if the implemented bots are working well. To do it, open a new terminal and run the file "run_bots.sh". In the terminal where you have started the server, you can observe how the bots act, as well as in the created world:

```
60619.
[21:18:11 INFO]: [RaspberryJuice] Opened connection to/127.0.0.1
:60649.
[21:18:11 INFO]: [RaspberryJuice] Starting input thread
[21:18:11 INFO]: [RaspberryJuice] Starting output thread!
[21:18:11 INFO]: [BuilderBot]: Building a house near you!
[21:18:11 INFO]: [BuilderBot]: House built!
[21:18:11 INFO]: [RaspberryJuice] Closed connection to/127.0.0.1
:60649.
[21:18:17 INFO]: [RaspberryJuice] Opened connection to/127.0.0.1
:60650.
[21:18:17 INFO]: [RaspberryJuice] Starting input thread
[21:18:17 INFO]: [RaspberryJuice] Starting output thread!
[21:18:17 INFO]: [InsultBot]: Type 'list_methods' in chat to see
what I can do!
[21:18:22 INFO]: [RaspberryJuice] Closed connection to/127.0.0.1
:60650.
[21:18:30 INFO]: [RaspberryJuice] Opened connection to/127.0.0.1
:60651.
[21:18:30 INFO]: [RaspberryJuice] Starting input thread
[21:18:30 INFO]: [RaspberryJuice] Starting output thread!
[21:18:30 INFO]: [OracleBot]: Ask me something!
[21:18:36 INFO]: [RaspberryJuice] Closed connection to/127.0.0.1
:60651.
>
```



You can also run each one separately using the command: **python examples/run_x_bot.py**

(where x is: tnt, oracle, insult, or builder)

Bot Descriptions

1. InsultBot

Functionality:

- Insults players in the Minecraft chat with random phrases chosen from a predefined list.

Features Highlighted:

- **Inheritance and Polymorphism:** This bot inherits from *MinecraftAgent*, reusing core functionality like sending messages (say) and the reflective method list_methods to dynamically inspect its capabilities.
- **Reflective Programming:**
 - The list_methods command allows players to inspect all available methods of the bot at runtime, demonstrating introspection.
- **Functional Programming:**
 - Uses *random.choice*, a pure function, to select an insult from the predefined list.

2. TNTBot

Functionality:

- Places TNT blocks near the player in Minecraft and announces their placement.

Features Highlighted:

- **Inheritance and Polymorphism:** Like all bots, TNTBot derives from *MinecraftAgent*, sharing core functionalities while implementing its unique perform_action method.
- **Reflective Programming:** It supports the list_methods command, showcasing its capabilities dynamically.
- **Functional Programming:**
 - TNT placement is computed dynamically using positional data (pos.x, pos.y, pos.z), and block operations rely on predefined functional APIs provided by the *mcpi* library.

3. OracleBot

Functionality:

- Answers player questions based on keywords in the question.
- Optionally logs interactions to a file for tracking.

Features Highlighted:

- **Inheritance and Polymorphism:** Implements perform_action and generate_response, building on the foundation provided by *MinecraftAgent*.

- **Reflective Programming:**
 - Uses a decorator `log_interaction` to enhance the `generate_response` method dynamically, enabling logging functionality without altering the core logic.
- **Functional Programming:**
 - The `generate_response` method processes questions using a functional mapping (`dict.items`) and matches keywords in a functional, declarative style.

4. BuilderBot

Functionality:

- Constructs a basic house near the player, using a predefined structure layout.

Features Highlighted:

- **Inheritance and Polymorphism:** Implements `perform_action` to leverage core Minecraft interactions while adding specific building logic.
- **Reflective Programming:**
 - Can list its capabilities dynamically via `list_methods`.
- **Functional Programming:**
 - The house structure is computed using a pure function `generate_house_structure` that defines block positions declaratively based on parameters like dimensions and material type.

Enjoy and implement something!